SIMATS ENGINEERING

PYTHON PROGRAMMING ASSIGNMENT 5

Data Acquisition, Cleaning, Analysis and AI Applications (CO5)

Student	Varshan J S
Register number	1924260130A
Institution	SIMATS Engineering
Course outcome	CO5 - Python libraries and tools for data acquisition, cleaning, analysis and introductory AI
Reference runtime	Python 3.13.5; pandas 2.2.3; NumPy 2.3.5; Matplotlib 3.10.8
GitHub repository	To be added after the Assignment 5 repository is created

Report scope

Part A presents the data-source review, database design, cleaning and normalisation rules, chart choices, debugging/profiling plan and AI application design. Part B implements the cleaning pipeline, SQLite storage, visualisations, adversarial tests, three-way aggregation benchmark, complaint AI and final engineering decision.

Data qualification: the assignment screenshots did not include official monitoring files. The implementation therefore generates reproducible synthetic readings for 12 Chennai-area station names over October 2026. No synthetic value is presented as an official Chennai air-quality measurement.

Assignment Structure and Case Study

Part / task	Title	Marks
A1	Design the Data Acquisition	7
A2	Design the Cleaning and Normalisation	5
A3	Design the Analysis and Visualisation	5
A4	Plan Debugging and Profiling	4
A5	Design an AI Application for Citizen Complaints	4
B1	Implement and Validate the Cleaning Pipeline	7
B2	Engineer Adversarial Data	5
B3	Loops vs Libraries vs SQL	8
B4	Final Engineering Decision	5

The simulated network includes 12 station identifiers and names: Adyar, Guindy, T Nagar, Anna Nagar, Velachery, Perungudi, Royapuram, Manali, Alandur, Kodambakkam, Nungambakkam and Thiruvottiyur. Each station sends 24 hourly observations per day for a 31-day month. PM2.5, PM10, NO2 and temperature are recorded. Nungambakkam and Thiruvottiyur are marked as Fahrenheit sources in the metadata.

A1 - Source inspection and storage design

The CSV contract is station ID, station name, ISO date, hour, PM2.5, PM10, NO2, temperature and temperature unit. Expected faults include missing or non-numeric readings, negative values, values above the sensor limit, duplicate station/date/hour keys, invalid date/hour values, station metadata mismatch, Fahrenheit temperatures, wrong delimiter, missing headers, unsupported encoding and empty files.

The SQLite design uses stations, readings, quality_log and daily_summary tables. `stations.station_id` is the primary key; readings have a foreign key and a unique key on station/date/hour; quality_log preserves file, line, reason and raw record; daily_summary stores min, max, mean and count. A composite index on station/date supports repeat station-month queries.

Representative SQL queries

```
SELECT station_name, AVG(pm25)
FROM readings JOIN stations USING(station_id)
WHERE station_name = ? AND reading_date >= ? AND reading_date < ?
GROUP BY station_name;

SELECT station_id, reading_date, MIN(pm25), MAX(pm25), AVG(pm25), COUNT(*)
FROM readings GROUP BY station_id, reading_date;

SELECT hour, AVG(pm25) AS mean_pm25
FROM readings GROUP BY hour ORDER BY mean_pm25 DESC;
```

An indexed query locates the matching station/date range in approximately O(log n + k), where k rows are read for aggregation. Reloading and parsing all CSV files for every question requires an O(n) scan and memory for the loaded data. SQLite adds database and index storage but is preferable for repeated analysis.

A1 acquisition algorithm

```
ALGORITHM AcquireAndStoreStationFiles
BEGIN
  INPUT raw folder and station metadata; connect to SQLite
  CREATE tables if absent and load station metadata
  FOR EACH daily CSV file in sorted order
    OPEN UTF-8 file and validate required header
    FOR EACH record with line number
      VALIDATE station, date, hour and measurements
      CONVERT Fahrenheit temperature to Celsius
      IF primary PM2.5 or key is invalid THEN log and continue
      IF station/date/hour duplicate THEN log and continue
      INSERT values with parameterised SQL
    END FOR
  END FOR
  COMMIT and close database
  OUTPUT counts and quality log
END
```

Part A - Cleaning, Normalisation and Visualisation

A2 - Cleaning rules

Condition	Rule	Reason
Missing PM2.5	Drop row; log source and reason	PM2.5 is the primary measure and should not be fabricated
Missing PM10 / NO2	Station-specific median imputation	Retains auxiliary coverage while limiting outlier influence
Missing temperature	Forward-fill within station; leading gap uses station median	Uses local time continuity without crossing station boundaries
Negative / above-999 pollutants	Reject primary PM2.5; invalid auxiliary channels set NULL then imputed	Treats impossible sensor values as faults
Duplicate key	Keep first station/date/hour record; log later duplicate	One observation per station and hour
Fahrenheit reading	Convert using (F - 32) x 5/9	Stores temperature consistently in Celsius

PM2.5 min-max normalisation uses (x - min) / (max - min). Z-score uses (x - mean) / population standard deviation. Both are calculated per station; the raw PM2.5 value remains available for original-unit reporting. Constant ranges map to zero to avoid division by zero. Dates use YYYY-MM-DD, station names use the metadata spelling, numeric readings are REAL, missing values are SQL NULL and temperature is stored in Celsius.

A2 cleaning algorithm - key steps

```
ALGORITHM CleanAndNormaliseStationReadings
BEGIN
  GROUP readings by station and sort each group by date/hour
  FIND station medians for PM10, NO2 and temperature
  FOR EACH reading
    IF PM2.5 missing, negative or above 999 THEN log and remove
    IF PM10 or NO2 missing/impossible THEN median-impute and log
    IF temperature missing/impossible THEN forward-fill or use median
  END FOR
  COMPUTE station PM2.5 min, max, mean and standard deviation
  CALCULATE min-max and z-score for retained readings
  OUTPUT cleaned data and quality log
END
```

A3 - Chart design

Question	Chart	Why it fits
Monthly PM2.5 trend	Line chart	Shows daily direction, spikes and changes over time
Station comparison	Bar chart	Ranks station mean values
Temperature vs PM2.5	Scatter plot	Shows association, clusters and outliers, not causation
Hours with worst air	Box plot by hour	Compares median, spread and outliers for each hour

The daily summary table contains station/date, PM2.5 minimum, maximum, mean and valid observation count. It is produced with SQL GROUP BY and stored for repeated queries. A Python loop and SQL aggregation both take O(n) time; SQL can reduce application-side memory and the summary table speeds repeated reports at the cost of storage.

A3 daily summary algorithm

```
ALGORITHM BuildDailyStationSummary
BEGIN
  GROUP cleaned readings by station and date
  FOR EACH group
    CALCULATE minimum, maximum and mean PM2.5
    COUNT valid readings and store summary row
  END FOR
  OUTPUT daily summary table
END
```

Part A - Debugging, Profiling and AI Design

A4 - Debugging plan

Use temporary print statements to inspect a few raw rows, parsed data types, rejection counts and station/date keys. Add assertions for hour bounds, PM2.5 range, foreign keys and uniqueness. Use the logging module with DEBUG for row-level diagnosis, INFO for batch totals and WARNING/ERROR for invalid records and file failures. Use pdb or the VS Code debugger to step through parsing, conversion, normalisation and SQLite insertion. Preserve malformed examples as regression tests.

A4 - Profiling plan

Measure acquisition, cleaning, database insertion, aggregation and chart rendering separately with time.perf_counter(). Use timeit for small functions and cProfile for the plain-Python aggregation. Compare repeated runs against the same input and verify output equality before comparing speed. Thousands of print calls add formatting and terminal I/O overhead; logging levels can be raised to WARNING or ERROR in production.

A5 - Complaint AI design

The lexicon baseline adds +1 for a positive token and -1 for a negative token. An immediately preceding negator flips the next opinion token; two consecutive negators cancel by parity. Positive totals are labelled positive, negative totals negative and zero neutral. The extractive summariser splits complaints into sentences, removes common words, counts informative-word frequency and selects sentences with the strongest frequency score.

The rule-based chatbot identifies a station and measure (PM2.5, PM10, NO2 or temperature), selects an explicit date or the latest date in the sample database, and runs a parameterised SQL query. It must label that date as the latest sample date rather than the actual current day. Sarcasm, mixed opinions and implicit context are limitations. A trained Naive Bayes or language model could improve contextual classification after collecting local labelled examples, but would need model memory, evaluation and monitoring; uncertain complaints should remain subject to human review.

```
ALGORITHM AnswerCitizenAirQualityQuestion
BEGIN
  INPUT citizen question
  IDENTIFY pollutant, station and date (or latest stored sample date)
  IF required field cannot be identified THEN return clarification
  EXECUTE parameterised SQL query
  IF no reading exists THEN return no-data message
  ELSE return average, date and unit
END
```

Part B - Implementation and Experimental Investigation

B1 - Cleaning pipeline and validation

The generated dataset contains 12 stations, 31 days and 24 nominal hourly observations per station/day. The code validates required CSV headers, date/hour fields, station identity and sensor ranges. Duplicate keys are rejected, Fahrenheit readings are converted to Celsius, missing PM2.5 rows are dropped, auxiliary gaps are imputed and all actions are written to a quality log. Cleaned readings are stored in SQLite and a daily summary is computed with SQL aggregation.

Measure	Result
Station count	12
Daily station files	372
Raw rows including injected duplicates	8,931
Clean rows stored	8,873
Quality events	150
Daily summary rows	372

The small completed-code example returns Adyar 48.0, Guindy 95.0 and Guindy 121.0. The normalised values are 0.000, 0.644 and 1.000. A 1205 test value is skipped. SQL results for the full synthetic month: Adyar mean PM2.5 is 48.01; 372 daily station summaries were created; the top network hours are 08:00 (85.02), 18:00 (84.88) and 19:00 (84.72). Manali has the highest station mean (100.81). These are properties of the generated test sample only.

CSV loop vs pandas cleaning benchmark

Input rows	Valid	CSV time	Pandas time	CSV result MB	Pandas frame MB
1,000	990	0.0014s	0.0053s	0.031	0.128
100,000	98,865	0.1369s	0.0335s	3.027	12.823
1,000,000	988,468	1.3026s	0.2153s	30.682	128.204

Both implementations retained the same valid-row count. CSV loops were faster for 1,000 rows, while pandas was faster at 100,000 and 1,000,000 rows. The pandas DataFrame used more memory in this experiment; a chunked CSV pipeline is an option when RAM is constrained.

B1 - Visualisation Evidence

The chart inputs come from cleaned SQLite readings. The data is simulated, and the units are shown as generated measurements rather than official monitoring outputs.

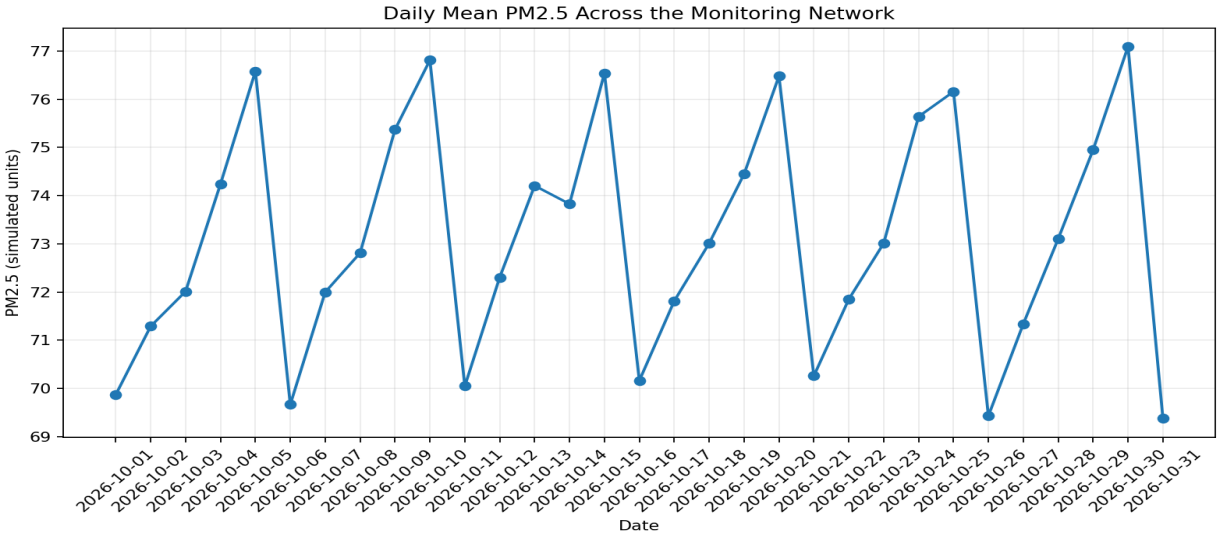


Figure 1. Daily mean PM2.5 over October 2026. The line shows recurring variation from day to day; the pattern reflects the deterministic test-data generator and should not be interpreted as a real city trend.

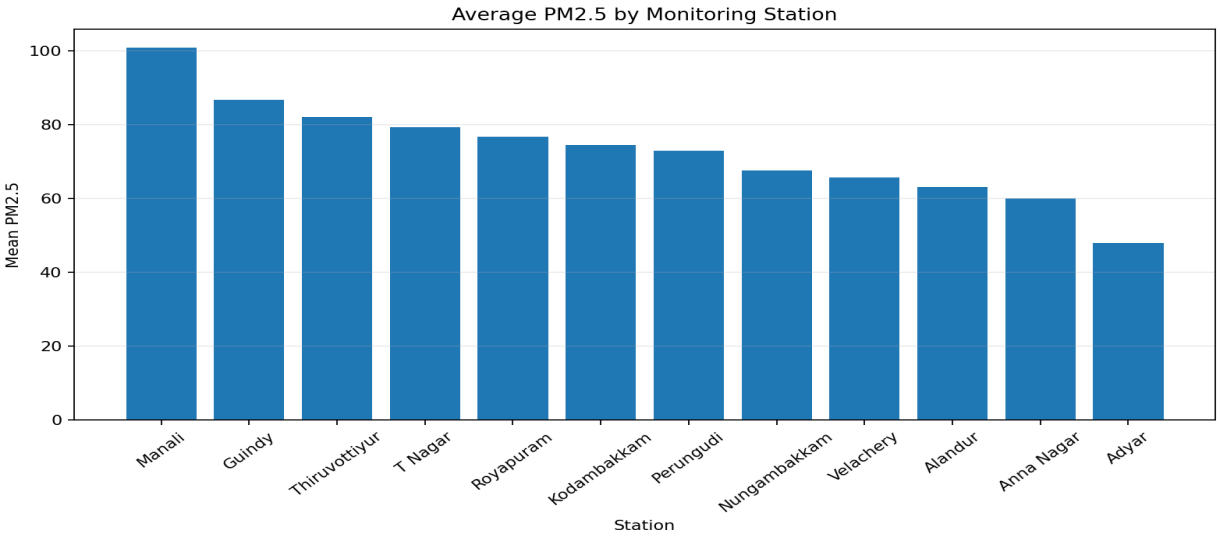


Figure 2. Station mean PM2.5. Manali ranks highest in the simulated sample. The bar chart supports station comparison but does not establish why the readings differ.

B1 - Additional Visualisation Evidence

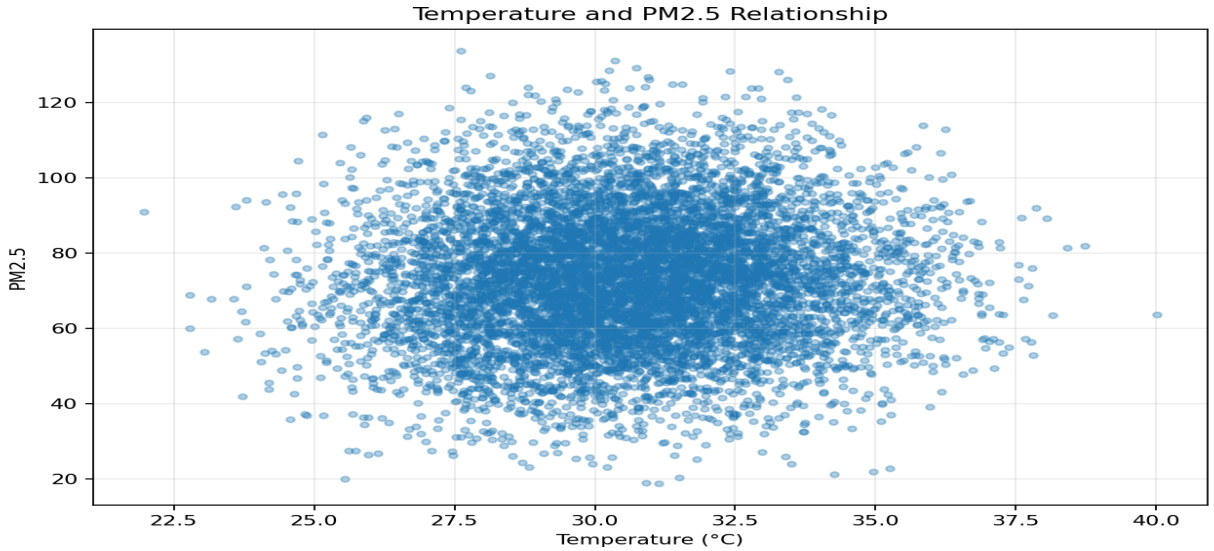


Figure 3. Temperature versus PM2.5. The scatter plot shows the spread and possible association between variables; correlation does not imply that temperature causes a change in PM2.5.

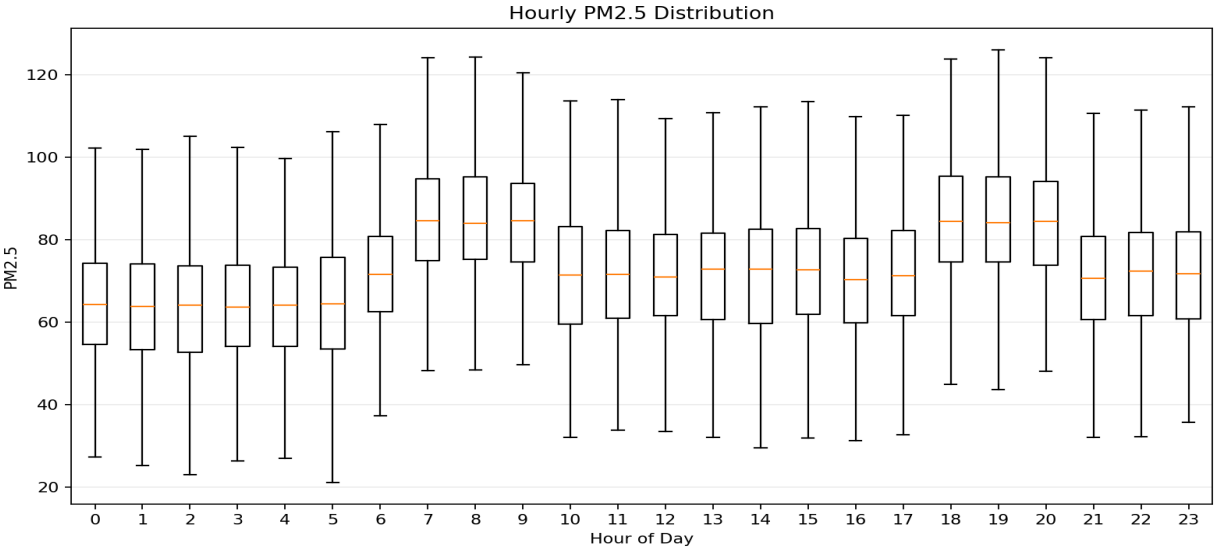


Figure 4. Hourly PM2.5 distribution. Comparing medians and spread indicates how the hourly profile varies. 08:00 had the highest average hour in the generated data.

B2 - Adversarial Data and SQL Safety

Adversarial input	Observed handling	Design lesson
Missing CSV column	Rejected with exact missing-column list	Validate schema before processing
Wrong delimiter	Header parsed as one field; required columns absent	Reject rather than silently shift fields
UTF-16 encoding	Rejected as unsupported UTF-8 input	Declare and validate file encoding
Empty file	Rejected as empty/no header	Do not treat empty input as an empty successful batch

The SQL injection value `Adyar' OR '1'='1` made the string-concatenated demonstration query return all three rows. The parameterised query treated the value literally and returned zero rows. The completed TODO uses one executemany call for all inserts and a question-mark placeholder in the station lookup; the verified averages are Adyar 54.0 and Guindy 108.0.

The sentiment adversarial set contains sarcasm, double negation and mixed opinions. The simple word-list approach is transparent but does not fully understand sarcasm and sentence contrast. The B4 test on 50 examples is used to quantify its agreement with the reference labels.

B3 - Three-way aggregation benchmark

Stations	Rows	Python loop	pandas	SQLite total	SQL query
1	8,760	0.0026s	0.0023s	0.0111s	0.0025s
12	105,120	0.0326s	0.0066s	0.1835s	0.0332s
120	1,051,200	0.3576s	0.0527s	1.5994s	0.4395s

All three methods agreed for 8,760, 105,120 and 1,051,200 readings. Pandas groupby was fastest for the largest in-memory batch. SQLite end-to-end includes loading rows; query-only time is lower and becomes attractive when the data is already stored. The cProfile report identifies the plain Python loop and repeated dictionary lookups as its main profiled work. The 120-station NumPy input arrays used about 10.03 MB.

B4 - Complaint AI and Final Engineering Decision

B4.1 - Sentiment and extractive summary

The lexicon scorer evaluated 50 generated complaints and agreed with the reference labels on 42/50 examples (84.0%). The disagreements are mainly mixed-opinion sentences where the word list counts positive and negative terms without understanding the contrast. `outputs/complaint_predictions.csv` contains each reference label and prediction; `outputs/complaint_summary.txt` compares the top-frequency sentence selection with the first-k baseline.

A rule-based chatbot answers station/pollutant/date questions by issuing a parameterised SQLite query. It reports the latest available sample date when the question says 'today'; it does not claim that the synthetic October data is real-time. Lexicon AI is a useful transparent triage baseline, but uncertain, sarcastic and health-related complaints require human review.

B4.2 - Composite index experiment

Measurement	Reference result
Mean query time without index	32.168 ms
Mean query time with index	0.082 ms
Index build time	0.305 s
Observed speed-up	394.2x

Without the index, SQLite scans the benchmark readings table. With the composite (station_id, day) index, the query plan searches the selected station/date range. The index consumes storage and increases update cost, but it substantially reduces repeated station/month query time on the test database.

B4.3 - Production decision

At 112 stations reporting every 15 minutes, the board would receive 10,752 readings each day and approximately 3,924,480 per year. Fifty thousand complaints per month is about 1,667 each day. Keep raw CSV files immutable for traceability; store cleaned readings and quality events in a relational database; create daily/monthly summary tables; use batched writes and a composite station/date index. SQLite is appropriate for this single-user prototype, while a concurrent production service should evaluate PostgreSQL or another server database.

Use the lexicon and extractive summary as a baseline, not as an automated authority. Collect locally labelled complaints, evaluate a trained classifier's precision/recall and class balance, and retain human review for uncertain or high-impact reports.

Overall Evaluation and Submission Notes

The project demonstrates the assignment workflow: acquire, inspect, clean, normalise, store, query, visualise, profile and interpret. It contains executable source modules, synthetic input files, an SQLite database, quality logs, summary outputs, charts, timing files, cProfile evidence and a README.

Task	Implementation result	Primary evidence
B1	8,873 cleaned rows; 150 quality events; 372 daily summaries	SQLite database, CSV outputs, four charts and 1k/100k/1m benchmark
B2	Four malformed-file tests rejected; injection neutralised by placeholder	Adversarial log and completed SQLite TODO output
B3	All loop/pandas/SQL group means agree up to 1,051,200 rows	Benchmark CSV and cProfile text
B4	42/50 sentiment agreement; measured index improvement	Prediction CSV, summary and index metrics

Complexity summary

CSV validation and cleaning are O(n) over n readings. Group aggregation is O(n) for Python, pandas and SQL. A running sum/count dictionary uses O(stations x days) state rather than retaining O(n) readings. A station/date index supports approximately O(log n + k) lookup, where k is the matching range read. Lexicon sentiment is O(w) in the number of complaint tokens; extractive ranking also orders sentence scores.

Reference environment: Python 3.13.5, pandas 2.2.3, NumPy 2.3.5 and Matplotlib 3.10.8 on the preparation environment. The student's Windows VS Code environment previously reported Python 3.14.2. Re-run the programs on that computer and record local timings before submission; performance values will differ between machines.

Submission reminder

Create a public or faculty-shared repository named `CSA08-1924260130A-Assignment5`, add at least one meaningful commit per task, and place the working GitHub URL on this cover page and in README.md. The assignment asks for genuine execution screenshots. The package includes text logs; capture terminal screenshots from the student's own VS Code runs and save them inside the corresponding task folders before uploading.

Conclusion

The design combines auditable data-quality decisions, reproducible inputs, relational storage, suitable visualisations, measured performance comparisons, safe SQL parameterisation and a clearly bounded complaint AI baseline. The results are supported by repeatable scripts and are explicitly qualified as synthetic test evidence rather than official environmental measurements.